

*Chapter No 6 – C Building Blocks***CE – 101 : C & P F***Computing and Programming
Fundamentals***Batch 2015**

Sir Syed University of Engineering & Technology
Computer Engineering Department
University Road, Karachi-75300, PAKISTAN

*Compiled By:**Syed Atir Iftikhar***CE - 101 : Computing and Programming
Fundamentals****■ Course Objectives:**

- Upon successful completion of this course, the student will be able to:
 - Fundamentals of Computer Engineering and Information Technology
 - Overview of History, Classification and Components of Computers
 - Number Systems and Logic Gates
 - Overview of Software, Operating Systems, Networks and Internet
 - Introduction about Programming
 - Basic Building Blocks, Loop, Decision Making

CPF Books

▪ Text Book:

1. **Introduction to Computers (7th Edition)**

By Peter Norton

2. **Turbo C Programming For The PC (Revised Edition)**

By Robert Lafore

▪ Reference Books:

1. **Computer, Communications and Information.**

By Sarah Hutchinson and Stacey Sawyer

2. **Let Us C**

By Yashavant Kanetkar

CPF - Chapter No 6 : Introduction to Turbo C Programming Environment

2
0
1
4
3

Marks Distribution

▪ Mid Term _____ 20

▪ Lab Work + Project _____ 20

▪ Quiz _____ 5

▪ Assignment + Class Performance _____ 5

▪ Semester Final Paper _____ 50

▪ **Total Marks** _____ **100**

▪ **Performance Bonus** _____ **5 Marks**

CPF - Chapter No 6 : Introduction to Turbo C Programming Environment

2
0
1
4
4

2

Course Instructors

- **Syed Aatir Iftikhar**

satirs@hotmail.com

Lecturer, CED

Room No: BF-03

2
0
1
4

7

CPF - Chapter No 6 : Introduction to Turbo C Programming Environment

Course Outline

✓ PART A

- History of Computer
- Classification of Computer
- Basic Components
- CPU, I/O, Peripheral Devices, Storage
- Von Neumann Architecture
- Number Systems
- Binary Numbers
- Boolean Logic
- Software
- Operating Systems
- Networks and Internet

✓ PART B

- Types of Programming Languages
- Algorithm
- Flow Chart
- Introduction to C Language Programming
- Basic Building Blocks
- Loops
- Decision Making Statements

2
0
1
4

8

CPF - Chapter No 6 : Introduction to Turbo C Programming Environment

4

CE – 101: Computing and Programming Fundamentals

Chapter**6****Batch 2014****C Building Blocks****Chapter 6 : Contents**

- Variables and Constants Types of Variables
- The scanf () function
- Address Operator
- The getch () function
- Different types of Operators
- Comments

**2
0
1
4**

10

CPF - Chapter No 6 : Introduction to Turbo C Programming Environment

5

Variables

- Variables may be the most fundamental aspect of any computer language.
- A variable is a space in the computer's memory set aside for a certain kind of data and given a name for easy reference.
- Therefore variables are used so that the same space in memory can hold different values at different times.

2
0
1
4

11

CPF - Chapter No 6 : Introduction to Turbo C Programming Environment

Defining a Variable in C

- Defining a variable in C
`int num ;`
- If there are more than one variable to be defined then they can be defined combined or separately. For example:
`int a, b, c;`
`int a;`
`int b;`
`int c;`

2
0
1
4

12

CPF - Chapter No 6 : Introduction to Turbo C Programming Environment

6

Basic Data Types in C Language

- C Language supports five basic data types.

Data Type	Description	Syntax	No of total Bytes occupied in memory
1. Character	Character data	char	1
2. Integer	Signed whole Integer	int	2
3. Float	Floating point number	float	4
4. Double	Double precision floating point number	double	8
5. Void	Valueless	void	0

CPF - Chapter No 6 : Introduction to Turbo C Programming Environment

2

0

1

4

13

Variable Definitions

- Variable are generally declared as:

type var-name ;

- Here 'type' is the data type and 'var-name' is the variable name.
- For example:

int number ;

CPF - Chapter No 6 : Introduction to Turbo C Programming Environment

2

0

1

4

14

7

Value Assignment to the Variable

- A programmer can assign a unique value to the variable.
- The general form of an assignment statement is

type var-name = value;

OR

var-name = value;

- For example:

int year = 2001;

number = 6;

2
0
1
4

CPF - Chapter No 6 : Introduction to Turbo C Programming Environment

15

What happens by writing the statement **int number = 6 ?**

- When you define a variable, the compiler sets aside an appropriate amount of memory to store that variable. In the above case we have specified an Integer variable, so the compiler will set aside 2 bytes of memory.
- This is large enough to hold numbers from 32768 to 32767 i.e 0 - 65535 for unsigned integers.

int number;

unsigned int number;

2
0
1
4

CPF - Chapter No 6 : Introduction to Turbo C Programming Environment

16

8

Long, Float and Double Data Type

- For situations when the normal integer is too small, the long integer (type **long** or **long int**) can be used. It occupies **4 bytes** of memory and can hold integers from $-2,147,483,648$ to $2,147,483,647$.
- There are also 2 kinds of floating point variables. One floating point variable type **float**, occupies **4 bytes** and can hold numbers from 10^{+38} to 10^{-38} with between 6 and 7 digits of precision.
- A double-precision floating point variable, type **double**, occupies **8 bytes** and can hold numbers from about 10^{+308} to 10^{-308} with about 15 digits of precision.

2014

17

CPF - Chapter No 6 : Introduction to Turbo C Programming Environment

Long Double, Int Data Type

- There is also a larger **long double** type. It occupies **10 bytes** and can hold numbers from about 10^{+4932} to 10^{-4932} .
- The **unsigned int** type holds numbers from **0 to 65,535**, rather than from $-32,768$ to $32,767$ as the regular **int** type does.
- There is no string variable type in C Language. Instead, strings are represented by arrays of characters.

2014

18

CPF - Chapter No 6 : Introduction to Turbo C Programming Environment

Program Using Variables

```
void main (void)
```

```
{
    int rollno;   char section;
    float gpa     = 3.25;
    rollno        = 100;
    section       = 'A';
    printf ( " My name is %s. My Roll no is %d. ", "Ahmad", rollno );
    printf ( " My section is %c and my GPA is %f. ", section, gpa );
}
```

Output:

My name is Ahmad. My Roll no is 100.
My section is A and my GPA is 3.25.

CPF - Chapter No 6 : Introduction to Turbo C Programming Environment

2
0
1
4

19

Field Width Specifiers

- The printf() gives programmer considerable power to format the printed output. By default floating point variable prints with 6 digits to the right of the decimal place.
- For example: **printf (" GPA = %f ", 3.25);**
- Output: **GPA = 3.250000**

CPF - Chapter No 6 : Introduction to Turbo C Programming Environment

2
0
1
4

20

10

Field Width Specifiers

- Field Width Specifiers controls how many characters will be printed following the decimal point.

- For example: `printf (" GPA = %02f ", 3.25);`

Output: GPA = 3.25

`printf (" GPA = %03f ", 3.25);`

Output: GPA = 3.250

CPF - Chapter No 6 : Introduction to Turbo C Programming Environment

2

0

1

4

21

Escape Sequences

- In C Language backslash symbol (\) is considered an "Escape" character: because it causes an escape from the normal interpretation of a string, so that the next character is recognized as having a special meaning.
- The following list shows the common escape sequence.

`\n` New line

`\t` Tab (move 8 characters forward)

`\b` Backspace

`\r` Carriage Return

CPF - Chapter No 6 : Introduction to Turbo C Programming Environment

2

0

1

4

22

11

Escape Sequences

\n	New line	2014
\t	Tab (move 8 characters forward)	
\b	Backspace	
\r	Carriage Return	
\f	Form feed (move to top of next page on the printer)	
\'	Single Quote	
\"	Double Quote	
\\	Backslash	
\xdd	ASCII code in hexadecimal notation	
\ddd	ASCII code in Octal notation (each d represents a digit)	

CPF - Chapter No 6 : Introduction to Turbo C Programming Environment

23

Program Using Escape Sequences

```
void main (void)
{
    clrscr();
    printf (" Hello students, \"Have a nice time.\" \n ");
}
```

Output: Hello students, " Have a nice time."

CPF - Chapter No 6 : Introduction to Turbo C Programming Environment

24

12

Scanf () Function

- In C Language, printf () is the output statement where as the scanf () is the input function statement.
- The scanf () function can accepts input to several variables in one statement.
- For example:
`scanf ( " %s %d", &name, &age );`
- & (Address Operator)

2014

Program Using Scanf ()

```
void main (void)
{
    float years, days;
    printf ( " Please type your age in years. " );
    scanf ( " %f ", &years );
    days = years * 365;
    printf ( " You are %.1f days old.", days );
}
```

Output: Please type your age in years. 10
 You are 3650 days old.

2014

Purpose of Address Operator

- Purpose of & Operator in C Language?
- *The C compiler requires the arguments of scanf () to be the address of the variables, rather than the variables themselves.*
- Memory of the computer is divided into addressed bytes, and these bytes are numbered from 0 to the upper limit of your memory (655357 if you have 640 K memory). These numbers are called the addresses of the bytes. Each variable occupies a certain location in the memory and its address is that of the first byte it occupies.

2

0

1

4

CPF - Chapter No 6 : Introduction to Turbo C Programming Environment

27

More about Address Operator

- Suppose we have, declared an integer variable, num in a program, and assigned it the value 2. If the program later refers to the name of the variable, num, the compiler will give the value stored in that variable, or 2.
- However, if you refer to the name of the variable preceded by the ampersand, &num, the compiler will give the address where num is stored. Here is the program that demonstrates this operation:

2

0

1

4

CPF - Chapter No 6 : Introduction to Turbo C Programming Environment

28

14

Program Using Address Operator

```
void main (void)
{
    int num;
    num = 2;
    printf ( " Value = %d, Address = %d ", num, &num );
}
```

Output: Value = 2, Address = 5528.

CPF - Chapter No 6 : Introduction to Turbo C Programming Environment

2
0
1
4

29

The getch () function

```
void main (void)
{
    char ch;
    print ( " Type any Charater: " );
    ch = getch ();
    printf ( " \n The character you typed was %c ", ch);
}
```

CPF - Chapter No 6 : Introduction to Turbo C Programming Environment

2
0
1
4

30

15

The getch () function

- You do not have to press the [Return] Key.
- It only takes a character.
- If you typed any wrong character, you cannot move backward to correct it.
 - get → get from outside
 - ch → character
 - e → echo (write)
- getch () vs scanf ()

2014

31

CPF - Chapter No 6 : Introduction to Turbo C Programming Environment

Operators

- Operators are words or symbol that causes a program to do something to variables.
- In C Language there are basically 4 types of operators.
 1. Arithmetic Operator
 2. Relational Operator
 3. Arithmetic Assignment Operator
 4. Increment Operator

2014

32

CPF - Chapter No 6 : Introduction to Turbo C Programming Environment

Arithmetic Operator

- C Language uses 4 types of Arithmetic Operator that are common in most programming languages and one the remainder operators which is not so common.

+	Addition
-	Subtraction
*	Multiplication
/	Division
%	Remainder

2014

Operator Precedence

- * and / operators are carried out before + and - .
- Programmers can alter the order of evaluation using parenthesis.
- Remainder Operator (%) is also called **Modulo Operator**.
- For Example:

$$\text{Answer} = 13 \% 5 = 3$$
- It is possible to include expressions involving Arithmetic Operator directly into printf () and other kinds of statements.

2014

Program

```
void main (void)
{
    int num;
    num = 2;
    printf ( " Number plus 4 is %d ", num + 4 );
    getch ();
}
```

2014

CPF - Chapter No 6 : Introduction to Turbo C Programming Environment

35

Arithmetic Assignment Operator

- These operators are specially for C programmers.
- With the help of these operators, programmer can compress programming statements.
- There are 5 types of Arithmetic Assignment Operator

+	=	Addition Arithmetic Assignment Operator
-	=	Subtraction Arithmetic Assignment Operator
*	=	Multiplication Arithmetic Assignment Operator
/	=	Division Arithmetic Assignment Operator
%	=	Remainder Arithmetic Assignment Operator

2014

- For Example: **total = total + number;**
 total += number;

CPF - Chapter No 6 : Introduction to Turbo C Programming Environment

36

18

Program Arithmetic Assignment Operator

```
void main (void)
```

```
{
    int total = 0;   int count = 10;
    printf ( " Total = %d \n ", total );
    total += count;
    printf ( " Total = %d \n ", total );
    total += count;
    printf ( " Total = %d \n ", total );
}
```

Output:

Total = 0 Total = 10 Total = 20

2
0
1
4

37

CPF - Chapter No 6 : Introduction to Turbo C Programming Environment

Increment Operator

- C Language uses another operator that is not common in other languages i.e the **Increment Operator**.
- For Example:
num ++ ; or num -- ;

2
0
1
4

38

CPF - Chapter No 6 : Introduction to Turbo C Programming Environment

Program Using Increment Operator

```
void main (void)
```

```
{
    int num = 0;
    printf ( " Number   =   %d \n ", num );
    printf ( " Number   =   %d \n ", num + + );
    printf ( " Number   =   %d \n ", num );
    getch ();
}
```

Output: Num = 0 Num = 1 Num = 1

CPF - Chapter No 6 : Introduction to Turbo C Programming Environment

2
0
1
4

39

Relational Operator

- Relational Operator checks the relation about the variables.
- The output of the relational operator will be 1 if the relation is true and 0, if the relation is false.
- In C Language, there are 6 Relational Operators:

<	Less than
>	Greater than
< =	Less than or Equal to
> =	Greater than or Equal to
= =	Equal to
! =	Not Equal to

CPF - Chapter No 6 : Introduction to Turbo C Programming Environment

2
0
1
4

40

20

Program Using Relational Operator

```
void main (void)
{
    int age ;    age = 15;
    printf ( " Is age less than 20 ? %d \n ", age < 20 );
    age = 30;
    printf ( " Is age less than 20 ? %d \n ", age < 20 );
    getch ( );
}
```

Output: Is age less than 20 ? 1
 Is age less than 20 ? 0

2
0
1
4

CPF - Chapter No 6 : Introduction to Turbo C Programming Environment

41

Precedence between Operators

```
void main (void)
{
    printf ( " Answer is %d ", 1 < 2 + 4 );
}
```

Output:

Answer = 5 (Wrong)
 Answer = 1 (Right)

- Note
- Arithmetic Operators have higher precedence
 i.e they are evaluated before the Relational Operators.

2
0
1
4

CPF - Chapter No 6 : Introduction to Turbo C Programming Environment

42

21

Comments

- It is helpful to be able to put comments into the source code that can be read by humans but are visible but are invisible to the compiler.
- A Comments begins with the two-character symbol slash-asterisk (/*) and ends with an asterisk-slash (*/).

2014

Program Using Comments

/* Program displays the name and age of the student */

```
void main (void)
{
    int age = 25;
    printf ( " Name is %s. My Age is %d. ", "Ahmad", age );
    getch();
}
```

Output: Name is Ahmad. My Age is 25.

2014

Class Assignment No 2

1. What do you mean by variable definition and variable assignment. Give examples.
2. What happens by writing the statement `int number = 6`
3. Difference between `\n`, `\r`, `\f` and `\t`.
4. Write a program that take the input of your age and then calculate the total number of hours old.
5. What is the purpose of `&` ?
6. Differentiate between `getch ( )` and `getche ( )` with example.
7. Write a program that takes a user marks for the 5 subjects and then calculate the total marks obtained by the user.
8. Write a program using: `==`, `++`, `%` and `<=` operators.

2

0

1

4